BELEM CAPITAL / DEFI DEPENDENCY RISK

# Doomberg Terminal

## Project manual and defense guide

Understand the investment.
Trace its dependencies.
Explain the risk.
Defend the decision.

A complete plain-language guide to the checked prototype: operation, mathematical methods, worked examples, live demonstrations and 28 hard questions with answers.

Version: vinkas / e85961f
Prepared: 11 October 2026
Audience: project team, coaches and technical judges

Read-only research and documentation. No trades executed.

START HERE

# How to read this manual

This is an explanation of the finished prototype on **vinkas, commit e85961f**, checked on 11 October 2026. It is written for someone who knows basic engineering but is new to DeFi. It describes actual behavior, including the assumptions you must be able to defend.

## Three kinds of example

| Label | What it means |
|---|---|
| Live snapshot | Read from real protocol APIs and blockchain contracts. Values are time-stamped and will change. |
| Controlled demonstration | Real discovered dependencies, with a deliberately fixed probability or exit cost. It demonstrates reasoning, not a genuine prediction. |
| Worked arithmetic | Small invented numbers used to understand an equation. These are not market quotes. |

## Reading routes

**To understand the product:** read pages 3-11, then the live example and demo script on pages 28-30. **To explain the mathematics:** read pages 12-25 and work through pages 26-27. **To prepare for judging:** finish with the questions and answers on pages 34-40.

## Contents at a glance


| Pages | Topic |
|---|---|
| 3-9 | Purpose, finance terms, architecture and discovery |
| 10-11 | Dashboard and monitoring instructions |
| 12-25 | Probability, simulation, propagation, exits and calibration |
| 26-30 | Worked examples, live snapshot and demonstration script |
| 31-33 | Operations, strengths, evidence and rehearsal checklist |
| 34-40 | Critical questions and answers |


The strongest explanation is a traceable claim: what was observed, what was assumed, how damage propagates, and what decision follows. Do not describe a modeled probability as a measured fact.

01 / PURPOSE

# What we built, and why

Doomberg Terminal is a decision-support dashboard for Belem Capital. It examines a DeFi investment, finds the things that investment relies on, estimates the loss if one fails, and compares that risk with the cost and feasibility of leaving.

## The problem in ordinary words

A position can look like one token on a balance sheet but depend on many separate systems. A lender might own a USDC claim whose borrowers posted a Pendle principal token. That token depends on an underlying asset, an issuer, an oracle, a trading pool, protocol contracts and a settlement chain. Failure several steps away can damage the position.

An incident flag alone is not enough. A price error may stay inside the lending safety buffer. A serious warning may still be cheaper to bear than an expensive exit. A recommendation to sell is useless if liquidity cannot support the sale.

| Question | What the terminal supplies |
|---|---|
| What do we depend on? | An evidence-backed dependency graph. |
| What happens if it fails? | A distribution of modeled position losses and propagation paths. |
| Should we leave? | STAY / EXIT, costs, withdrawable share and reasons. |
| What changes over time? | Scheduled screenings, stored forecasts, alerts and history. |

**Failure modes** include depeg (lost reference value), exploit (abused code), unbacked minting (unsupported tokens), issuer default (lost backing), redemption freeze (blocked access), liquidity drain (removed cash), and chain halt. Wrong prices and stale updates are separate oracle modes.

## Why the approach is useful

Discovery connects an observation to an economic exposure. Simulation turns that exposure into loss. Exit pricing makes the recommendation operational. The interface makes the chain of reasoning visible, so an employee can challenge it instead of accepting an unexplained score.

It is a working research prototype. It does not manage private keys, execute trades, guarantee profits, prove reserves, or provide a calibrated production risk service. The manual does not make claims about Belem Capital's actual revenues or trading book.

02 / FINANCE WITHOUT JARGON

# The investment: tokens, claims and PTs

| Term | Plain meaning |
|---|---|
| Token | A blockchain-recorded asset or claim. Ownership alone does not prove the claim can be redeemed. |
| USDC / USDT | Dollar-referencing tokens from issuers. They can trade away from $1 and have issuer and redemption risks. |
| Underlying asset | The asset that supports a token's economic value. A wrapper or PT can depend on another token. |
| PT: principal token | Pendle separates principal from future yield. A PT is a maturity-dated claim on a defined redemption amount. |
| YT: yield token | The separate claim to yield before maturity. The terminal does not analyze YT positions. |
| Wrapper / vault share | A token representing a staked or deposited asset. Its own discount risk differs from the underlying's loss. |

## A simple PT example

Imagine paying $95 today for a claim that is intended to redeem a unit worth $100 at maturity. The $5 difference is the return if redemption works as expected. Before maturity, the resale price may fall. If the backing fails, the maturity claim can also be impaired. This is a worked example, not a promise of $100.

Pendle PTs are denominated in a specified accounting asset. Redemption is not necessarily one unit of the yield-bearing wrapper or one US dollar. Check the market's denomination; PT-ezETH, for example, represents ETH-denominated principal rather than one ezETH share. [P1]

## Why PT-reUSD is not just reUSD

The live example has three different assets: PT-reUSD-10DEC2026 as collateral, reUSD as discovered backing, and USDC as the lender's loan asset. reUSD is a yield-accruing asset in this project; a price above $1 is not by itself proof of a broken dollar peg.

The two supported position types are: **hold a PT**, or **supply to a Morpho market whose collateral is a PT**. These are different economic exposures.

02 / FINANCE WITHOUT JARGON

# The machinery around the investment

| Term | What it does / why we care |
|---|---|
| Lending market | Lenders supply a loan asset; borrowers post collateral and borrow. A lender owns a claim, not the borrower's collateral. |
| LTV | Loan-to-value: debt divided by collateral value. Debt $80 / collateral $100 means 80% LTV. |
| LLTV | Liquidation loan-to-value: the market's threshold for liquidating a borrower. It is not the lender's loss rate. [P2] |
| Liquidation | Selling or seizing collateral to repay an unsafe loan. Failure to do it in time can leave bad debt. |
| Utilization | The fraction of supplied assets currently borrowed. At high utilization, little cash is immediately idle. |
| Oracle / feed | The oracle reports the price the lending protocol uses. A feed is one source it reads. A fresh answer can still be wrong. |
| AMM | Automated market maker: a pool with trading rules. A large sale can move its price. |
| Slippage / price impact | The loss in execution value relative to a reference price. A quote can deteriorate before execution. |
| APY / TVL | Annual percentage yield / total value locked. Yield is compensation, not proof of safety; TVL is not cash available to exit. |
| NAV / drift | Net asset value is an accounting value. Drift here means modeled price movement while trading or liquidation is frozen. |

**Layers** are simply categories in the stack: settlement, backing, price information, sale liquidity and lending. They are not independent mathematical risk buckets. **Funding rate** is a payment between long and short perpetual-futures traders; it can help explain a yield source, but the terminal does not calculate a perpetual-futures funding strategy.

A market with 90% utilization has lent most of its supply. That does not mean 90% will be lost. It means withdrawal and loss propagation must be evaluated separately.

03 / SYSTEM MAP

# One complete trip through the system




**1 Discover:** start from the entered address and follow contracts, protocol APIs and curated backing information. **2 Monitor:** compare live readings with thresholds and stored readings. **3 Simulate:** sample possible failures and propagate their effects to our investment.

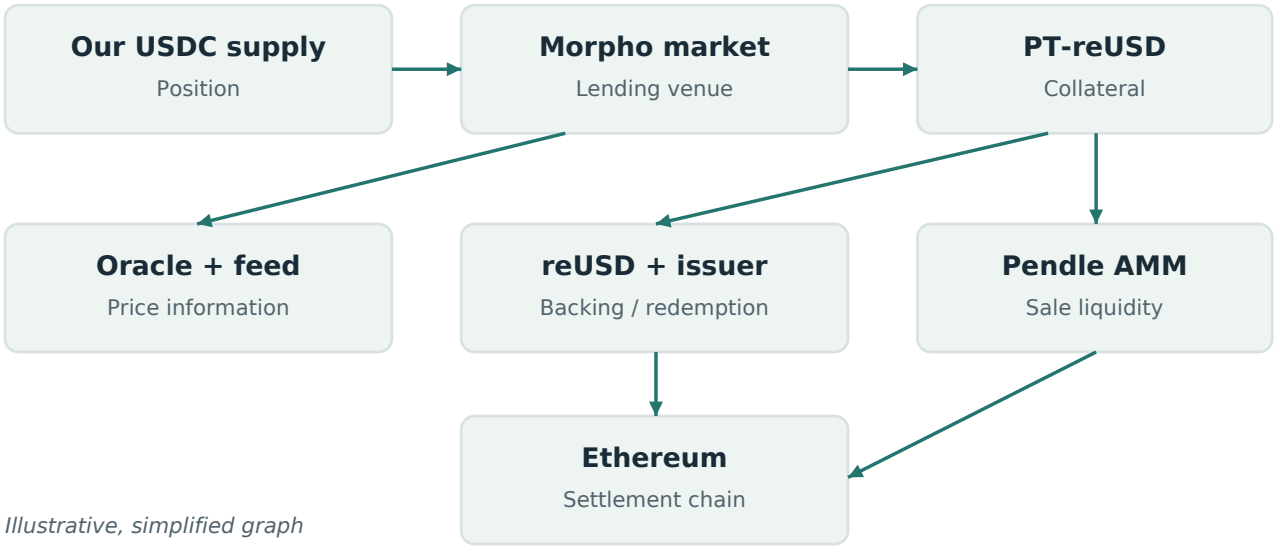
**4 Price exit:** get routes at the actual position size, check withdrawal liquidity, and add modeled costs. **5 Decide:** compare the value of avoiding losses with the cost of leaving. Explain the numbers and, for recorded monitoring runs, store forecasts and changes.

| Part | Responsibility |
|---|---|
| React frontend | Investment tabs, graph, scenario inspector, distributions, cost breakdowns and scheduled-monitoring panel. |
| FastAPI backend | Validates requests and exposes analysis, scenarios, replay, explanation and monitoring endpoints. |
| Python risk engine | Discovery, signal rules, probabilities, contagion, simulation, exit pricing and decisions. |
| SQLite + browser storage | SQLite persists recorded monitoring. Browser storage keeps open investment workspaces and saved scenarios. |

Python computes the numeric verdict. Optional AI writes an explanation of supplied results only. There is no autonomous trading agent and no signing transaction in this architecture.

03 / SYSTEM MAP

# Read a graph as a chain of obligations




*Illustrative, simplified graph*

A **node** is a component: an asset, market, feed, issuer or chain. An **edge** is a relationship such as holds, backed by, priced by or runs on. The graph's arrows point from the dependent to the thing it relies on. Damage normally travels back toward the dependent.

For example: our position deposits in a lending market; that market accepts the PT as collateral; the PT relies on reUSD. If reUSD loses value, the PT may weaken, borrowers may become unsafe, and our lending claim may suffer bad debt. The lender need not hold reUSD directly to be exposed to it.

## Two graph concepts that prevent confusion

**Depth** means how far a component lies from the investment in the discovered stack. It does not mean its probability is higher. **Risk share** means its share of the model's expected-loss contributions, not its probability of failing.

Visual groups declutter the display. They do not remove dependencies from the analysis. Expand a group or use the complete view to inspect the underlying nodes and evidence.

04 / DEPENDENCIES

# Which dependencies the project models

| Category | Nodes / role |
|---|---|
| Position | The entered holding or lending claim and its size. It is the final recipient of modeled damage. |
| Assets | PT collateral or holding, backing tokens, loan token, and any discovered wrapper chain. |
| Lending markets | Our own market plus other Morpho collateral venues where selling the same PT can depress its price. |
| Oracle + feed | Oracle contract and its base / quote price feeds, or a discovered exchange-rate source. |
| AMM | Pendle market where the PT trades; liquidity, yield and expiry influence selling and fire-sale effects. |
| Protocols | Pendle V2 and Morpho Blue contract systems, with code-risk assumptions. |
| Issuers / custodians | Entities expected to honor backing or redemptions, often supplied by the registry. |
| Mint / upgrade controls | EOA keys, multisigs, timelocks, proxies or role-based controls that can change or create tokens. |
| Bridges | A bridged token's dependency on locked backing and message / mint security, where identified. |
| Settlement chain | The network whose transactions, liquidation and withdrawals must continue. |

An **EOA** is an externally owned account controlled by a private key. A **multisig** requires several authorized signatures. A **timelock** delays privileged actions. An **upgradeable proxy** can route calls to replacement code. Identifying these structures is evidence about control, not a complete security audit.

There is no fixed list of 48 dependencies for every investment. Graph size changes with the position, the data available and the discovery limits. The checked live market had 17 nodes and 20 edges.

04 / DEPENDENCIES

# How dependencies are actually discovered

## Follow contracts and protocol information

For a token position, the input must be a Pendle PT. For a market position, discovery checks that its collateral is a PT. Morpho supplies market and token metadata. Pendle identifies the PT's pool, expiry and backing. RPC calls inspect feeds, token contracts and control structures. ERC-4626 asset() calls can reveal a vault's underlying token.

| Source | Typical evidence |
|---|---|
| Morpho API | Loan/collateral tokens, LLTV, utilization, supply, borrowing, idle liquidity, oracle price and collateral venues. |
| Pendle API / SDK | PT pool, underlying, maturity, implied APY and size-specific swap route. |
| Blockchain RPC | Feed answer / update age, code, proxy / ownership and mint-control reads; block references where available. |
| DefiLlama / KyberSwap | Independent token USD marks and route output / execution costs. |
| Registry / heuristic | Issuer, backing, asset class, bridge labels, or inferred relationships where direct reads cannot prove them. |

## Discovery is bounded, not exhaustive

The asset recursion has a configured depth limit. Collateral-venue discovery models the largest five qualifying Morpho markets, each with at least $250,000 posted collateral, excluding our own market. Smaller qualifying markets beyond the cap raise a warning. Other lending protocols and private/off-chain dependencies are not exhaustively searched.

Each node and edge carries source evidence. A symbolic backing token, a name-based classification or a curated issuer relationship is weaker evidence than a successful contract read. The operator should inspect source labels and discovery warnings, especially for unknown assets.

Oracle market-tracking classification uses feed descriptions and discovered structure. It is a heuristic, not verified behavioral testing. The current “oracle vs market” comparison uses Morpho-provided token marks, not a separate continuously sampled DEX oracle.

05 / OPERATOR GUIDE

# Use the dashboard: research and scenarios

| Step | What to do and what it means |
|---|---|
| 1 Open investment | Choose a preset or enter a PT address / PT-collateral market ID, chain, position size, book size and horizon. |
| 2 Read the snapshot | Check discovery warnings, observation age, withdrawable share and the monitoring snapshot. Refresh after a server restart or for new live readings. |
| 3 Use left-side tabs | Keep several investments open. Selecting a tab changes the active investment while preserving its analysis and scenario history. |
| 4 Inspect a component | Select a graph box or modeled risk contributor. Read the evidence, classification and available failure modes. |
| 5 Test a scenario | Choose a failure mode and Signal strength. Automatic uses the snapshot's detected signals; manual levels create a what-if input. |
| 6 Read all result tabs | Summary, distribution, exit plan and assumptions explain the verdict, range, costs and causal paths. Inspect the path in the graph. |
| 7 Optional AI summary | Generate it only when needed. Without a configured provider key, the explanation uses a template. |

**Advanced overrides:** the probability slider fixes p; the exit-cost slider fixes c. They are useful for controlled demos and sensitivity checks. They do not recalculate the discovered graph or make the fixed input a statistical estimate. Exit-cost overrides also do not create a route or increase the withdrawable fraction.

**Saved results:** an earlier scenario belongs to the snapshot that produced it. A refreshed graph can have different inputs and IDs. A browser-restored workspace must refresh its analysis before new scenarios, because the backend stores interactive graphs in memory.

The sidebar monitoring snapshot is the analysis you opened. It does not itself poll continuously. Use Scheduled monitoring for fresh recorded runs on a schedule.

05 / OPERATOR GUIDE

# Use the dashboard: scheduled monitoring

Open the **Scheduled STAY / EXIT** panel. It lists the watchlist, latest recorded verdicts, net avoidable loss versus exit cost, alerts, changes and timestamps. Expand an investment row to see recent history, main drivers and event messages.

| Control | Meaning |
|---|---|
| Monitor current investment | Save the current request as a watched position. Its size and horizon matter; this is not a wallet-verified holding. |
| Schedule | Choose a polling interval or pause. The API accepts paused (0) or at least 300 seconds, up to seven days. |
| Run now | Screen the watchlist once with current sources and store the run. This is not replay. |
| Remove | Remove a saved watchlist entry. Check the watchlist source afterwards. |
| STAY / EXIT / REVIEW | STAY and EXIT are modeled recommendations. REVIEW means required readings are missing or invalid; the numeric estimate is provisional. |

## What persists and what runs

Saved watchlist and interval live in SQLite. When no saved entries exist, the API falls back to a WATCHLIST JSON file if configured, otherwise the built-in presets. Removing the last saved position can therefore restore the fallback examples; it is not the same as pausing the schedule.

The monitor runs inside the API process and stops when that process stops. The loop plans from the last recorded run time; there is no external high-availability scheduler. While the panel is open it refreshes its status, more often during a run. A failed screening must not be mistaken for a new healthy result.

> The UI can store and show notifications. A configured webhook can post warning/critical events. Notifications inform an operator; they do not sign or submit an exit transaction.

06 / MODEL ROADMAP

# The mathematics in one picture

| Symbol | Definition / units |
|---|---|
| V | Entered position value, in USD. |
| B | Entered total book value, in USD. |
| H | Forecast horizon, in days; default 30. |
| lambda | Annual event rate, in events per component-year. |
| p | Chance of a specified failure during H, between 0 and 1. |
| L | Position loss fraction conditional on that failure. E[L] is its simulation average. |
| f | Instantly withdrawable / sellable fraction, between 0 and 1. |
| c | Exit cost as a fraction of the whole position, including modeled carry during redeployment. |
| E | Daily screening's sum of p_i × E[L_i] over components and modes. |
| Y | Daily yield credit over the remaining horizon; defined on page 23. |

## Three separate calculations

### Failure probability p / damage if it happens L / value of leaving

Probability answers **how likely?** Severity and propagation answer **how much do we lose if it happens?** The decision answers **is the loss we can avoid worth more than leaving?** A high probability with tiny damage can justify STAY; a low probability with severe damage can justify EXIT.

Example: a 10% chance of losing 40% gives 4% expected loss. For a $1,000,000 position that is $40,000. It does not mean the position is forecast to fall exactly 4%; it is an average over the modeled alternatives.

Always distinguish the failing component's severity from the final position loss. A 20% collateral drop does not necessarily produce a 20% lending loss, because collateral buffers and liquidation may absorb some of it.

07 / FAILURE PROBABILITY

# Probability: Gamma-Poisson, step by step

The starting model treats failure arrivals as a Poisson process: a class of components has an annual event rate lambda. That rate is uncertain, so it is represented by a Gamma distribution. Gamma is useful here because it describes positive rates and works naturally with event counts. [P3]

**lambda ~ Gamma(shape = k + 0.5, rate = T / m)**

**k** is a hand-set historical event count for a class; **T** is assumed component-years of exposure; **m** is the product of risk-factor multipliers. “Rate” in the Gamma parameterization is not lambda: a larger Gamma rate parameter means a smaller mean lambda. NumPy receives scale = 1 / rate.

The +0.5 comes from a Jeffreys-style prior for a Poisson rate. It prevents “no observed failures” from becoming “zero risk.” The engine's counts and exposure years are curated assumptions, not a fully audited dataset or numbers fitted from the short replay catalogue.

**P(at least one failure in H) = 1 - exp(-lambda × H / 365)**

## Worked example: fiat-stable depeg, no modifiers

The code uses k = 3 and T = 45. Mean annual rate = 3.5 / 45 = 0.0778. At that representative rate, 30-day probability = 1 - exp(-0.0778 × 30 / 365) ≈ 0.637%. The actual engine averages the transformed Gamma samples, so its mean need not equal this plug-in approximation.

The model draws a rate, transforms it to a horizon probability, and repeats. The 5th and 95th percentiles form a 90% modeled credible range. This captures uncertainty inside the chosen statistical assumptions; it does not capture every omitted dependency or prove real-world accuracy.

Risk factors include single-source feeds, thin collateral, control structures, issuer tier and young protocols. Some weights are large judgment calls. Longer horizons raise p; a hazard rate is not itself a 30-day probability.

07 / FAILURE PROBABILITY

# Signals update odds, not percentages

When a warning arrives, the engine updates probability through a likelihood ratio (LR). The statistical idea is Bayesian: evidence changes the odds of an event. The LR values in this prototype are hand-set rather than estimated from sufficient labeled observations.

**odds = p / (1 - p)**

**updated odds = prior odds × LR**

**updated p = updated odds / (1 + updated odds)**

## Worked example with a 1% prior

| Level | Depeg LR | Updated probability |
|---|---|---|
| None | No update | 1.00% |
| Watch | 20 × 0.25 = 5 | 4.81% |
| Alert | 20 × 1 = 20 | 16.81% |
| Corroborated | 20 × 5 = 100 | 50.25% |

A depeg alert has base LR 20. Oracle modes have base LR 40; other modes use their own values. A factor of 20 on **odds** is not “add 20 percentage points” and is not exactly “multiply probability by 20.” The formula keeps the result below 100%.

**Confirmed** is special: a person or replay sets it, and the code samples probability uniformly from 85% to 99%. It is a convention used for incident analysis, not the same odds update and not proof that a documented historical event is only 85% likely to have occurred.

A single watch is watch. Two different watch families become alert. Two families with at least one alert become corroborated. “Independent family” is a rule-based grouping; statistical independence is not established.

08 / MONITORING SIGNALS

# The actual automatic rules

| Reading | Watch | Alert |
|---|---|---|
| Peg deviation from known reference | 0.5% | 2% |
| Token / collateral price drop | 2% | 5% |
| PT implied-APY increase | 3 percentage points | 8 percentage points |
| Oracle above market mark | 2% | 5% |
| Utilization | 97% | 99.5% |
| Supply below recent high | 10% | 25% |
| AMM liquidity below recent high | 20% | 40% |
| Unrealized bad debt / supply | 0.1% | 1% |
| Feed stale | Not a watch rule | Staleness flag |

Change rules compare with recorded observations from the preceding 24 hours, excluding the last minute. They use the recent maximum or minimum, not necessarily a reading exactly 24 hours ago. Without stored observations, those change rules cannot run. Price-drop rules are limited to asset classes with assumed volatility at most 25% annually.

Feed staleness is currently based on 26 hours for ordinary feeds, or 35 days for identified NAV feeds. These are configured assumptions, not per-feed heartbeats verified from the provider. There is no built-in “0.5% deviation continuously for one hour” persistence rule.

Price drop and implied-APY jump share the price family. Utilization and supply outflow share the liquidity family. Bad debt uses solvency. This avoids treating two views of one movement as two independent confirmations.

The automatic price-gap rule only flags oracle overpricing. It does not yet flag oracle underpricing, even though underpricing at another collateral venue can trigger the modeled fire-sale channel. The prototype also does not comprehensively detect hacks from news, monitor RPC latency, or label every protocol downtime event.

08 / MONITORING SIGNALS

# Contagion: remembering recent stress

Failures can cluster. An event elsewhere may signal elevated conditions for related components. The prototype uses a Hawkes-style multiplier with exponential decay. This is a heuristic multiplier on a sampled rate, not a fitted Hawkes process with learned excitation parameters.

**multiplier(t) = min(5, 1 + sum of alpha_i × exp(-age_i / 30 days))**

| Event relationship | alpha |
|---|---|
| Recorded outcome on same component | 3.0 |
| Outcome on same class and failure mode | 1.0 |
| Other recorded outcome | 0.2 |
| Alert episode on same class and mode | 0.3 |
| Other alert episode | 0.05 |

## Worked decay example

An outcome on another synthetic stablecoin five days ago adds exp(-5/30) = 0.846 to the same-class hazard multiplier, making it about 1.846. Thirty days later the extra contribution has fallen substantially. The decay half-life is approximately 20.8 days. Events older than 120 days are omitted.

Recorded alert sightings are grouped into episodes. A gap over 36 hours starts a new episode. Combined alerts, including two independent watches that escalated, enter history. A current signal on a component is not counted again as its own alert contagion. A feed's warning shown on its parent oracle keeps the same causal origin.

The regime label uses the largest multiplier: calm below 1.2, elevated below 2, otherwise stressed. This summarizes model conditions; it is not an independently measured market regime.

These weights and the 30-day decay were not calibrated to a large incident dataset. They make the model responsive and explainable, but they can also transmit broad stress too widely. The multiplier is shown with reasons so it can be challenged.

09 / CONDITIONAL LOSS

# Monte Carlo: many possible outcomes

Instead of selecting one arbitrary loss, Monte Carlo repeatedly samples uncertain quantities and computes the resulting position loss. Each draw is one possible world. The collection gives an average, a median, a bad case and a histogram.

| Distribution | Role in the code |
|---|---|
| Gamma | Uncertain positive annual failure rate. |
| Beta | Loss fractions and shares between zero and one. |
| Exponential | Non-negative durations such as hours frozen. |
| Uniform | Borrower leverage and liquidation effectiveness inside chosen bounds. |
| Normal | Price-move noise, including around historical replay severity. |
| Lognormal | Positive stress widening of exit price impact. |

## A Beta example

**Beta parameters: alpha = mean × concentration; beta = (1 - mean) × concentration**

For a synthetic-stable loss prior with mean 35% and concentration 3, alpha = 1.05 and beta = 1.95. Low concentration makes the loss distribution broad. This does not mean every failure loses 35%; 35% is the distribution's average before propagation.

Screening uses 500 severity draws and 1,000 probability draws per component/mode. Detailed scenarios normally use 5,000 draws. Calm exit pricing uses its own draw set. Sampling more reduces numerical noise but cannot repair a wrong structural assumption.

The fixed random seed helps repeatability and comparisons with unchanged inputs. It is not a reason the real risks fail to change: live inputs, signal levels and stored events can change. A topology or draw-order change can still affect the sampled values.

09 / CONDITIONAL LOSS

# How damage travels to the position

| Relationship | Propagation in plain terms |
|---|---|
| PT backed by asset | Loss of backing reaches the PT. Wrapper / issuer / mint damage can flow through several hops. |
| Asset issued by entity | Default or frozen redemption can impair the token even without a contract exploit. |
| PT traded on AMM | Pool problems can create a resale discount; protocol exploit loss is attenuated for AMM paths. |
| Market accepts collateral | Collateral damage affects lenders through leverage, liquidation and bad debt, not a one-to-one token loss. |
| Market priced by oracle | Wrong or stale information changes borrowing and liquidation behavior. |
| Position denominated in loan token | If the loan asset depegs, its dollar value falls for the lender. |
| Settlement chain stops | Execution freezes; prices may drift and liquidators can miss their window. |

## Combine two modeled loss channels

**combined loss = 1 - (1 - loss A) × (1 - loss B)**

If A removes 20% and B removes 10% of the remaining value, combined loss is 1 - 0.8 × 0.9 = 28%. It is not 30%, because the second loss acts on what remains. The code uses this survival-fraction combination within a simulation draw.

The graph is evaluated dependencies first. The model distinguishes loss, oracle-price error, hours stale and hours frozen. It does not treat every graph edge as an identical percentage-loss link.

The survival product is a simplifying combination rule, not proof that causes are independent. Separately, daily screening sums expected-loss contributions across modes. That sum and P(any loss event) are approximations, not a complete correlated portfolio simulation.

10 / LENDING LOSS MODEL

# Why lenders have a collateral buffer

A borrower owes an amount D against initial collateral C. If collateral falls by d, its value becomes C × (1 - d). Lenders suffer a shortfall when available collateral cannot cover the debt. Liquidations can reduce that shortfall if prices are timely and execution is possible.

**borrower LTV ell = LLTV × Uniform(0.70, 0.97)**

**uncovered debt fraction = max(0, 1 - (1 - d) / ell)**

**bad-debt fraction after mitigation = uncovered fraction × (1 - effectiveness)**

## Worked example: LLTV 86%, leverage at 90% of threshold

ell = 0.86 × 0.90 = 0.774. A 3% collateral fall leaves 97% of initial value, more than 77.4% debt, so uncovered debt is zero in this simplified example. A 40% fall leaves 60% value: uncovered fraction = 1 - 0.60 / 0.774 = 22.48% of borrowed debt.

If liquidators prevent 50% of that shortfall, residual bad debt is 11.24% of borrowed debt. At 90% market utilization, the simplified loss on total supplied assets is about 10.12%. The actual simulation samples leverage and effectiveness, and may add other damage channels.

Ordinary liquidation effectiveness is sampled from 30-80%; abrupt exploits or mint/bridge shocks use 0-30%. Effectiveness is set to zero if the oracle does not track the market, is stale, or execution is frozen. The assumptions are explicit; actual borrower-by-borrower positions are not modeled.

86% is a teaching example, not the current example market's LLTV. The checked live PT-reUSD/USDC market reported 91.5%. Real protocol liquidation begins at LLTV; the equation above approximates eventual economic shortfall. [P2]

10 / LENDING LOSS MODEL

# Oracle errors, ratios and blind windows

## A wrong high price enables over-borrowing

**over-borrowing begins when LLTV × (1 + oracle overpricing) > 1**

At LLTV 86%, the error threshold is 1 / 0.86 - 1 = 16.28%. An error of 5% can raise an alert without crossing this drain threshold. An alert is evidence of a problem, not automatically a large lender loss or EXIT.

**drain = (1 - utilization) × max(0, 1 - 1 / (LLTV × (1 + error))) × Uniform(0.6, 1)**

This term only drains the idle share, not all supplied assets. It is added to the bad-debt channel. With 50% utilization, LLTV 86% and 100% overpricing, the factor before the last uniform term is about 20.93% of supply.

## Base and quote feeds must compose correctly

An oracle can represent a ratio: base price divided by quote price. Base-feed errors multiply the price factor; quote-feed errors divide it. A quote feed 15% too low makes the ratio 1 / 0.85 = 1.1765, or 17.65% too high. Simply changing the sign to +15% would miss the 86% LLTV threshold.

## Stale is different from wrong

**blind-window move ≈ |Normal(0,1)| × annual volatility × sqrt(hours / 8,760)**

When price discovery or settlement freezes, the model samples a downside movement during that window and disables timely liquidation. For a held PT, chain-halt drift is applied once at the position. A stale feed with a small drift inside the buffer can legitimately cause near-zero modeled lending loss.

Unknown wrong-price direction is sampled as overpricing 65% and underpricing 35%. Fixed replay severity uses the positive direction. Underpricing can harm a PT via forced sales in other venues even when it cannot over-borrow our lending market.

11 / OTHER LENDING VENUES

# The new channel: collateral fire sales

Other borrowers may post the same PT in other Morpho markets. An oracle error or real price fall there can trigger liquidators to seize and sell the PT. Their selling competes for the same Pendle pool and can depress the PT held by us or used as collateral in our market.

**seen value factor = (1 - actual drop if tracked) × (1 + oracle error)**

**liquidated share = clip((0.97 - seen value factor) / 0.27, 0, 1)**

The 0.27 is the width of the assumed 70-97% leverage range relative to LLTV. A stale or frozen venue is assumed unable to liquidate. The model bounds usable collateral by posted collateral and borrowing divided by LLTV × 0.835; 0.835 is the midpoint of that leverage range. It samples 50-100% of seized PT as sold.

**modeled dip = min(AMM impact of total forced sale, 1 - (1 + stressed APY)^(-years to expiry))**

Stressed APY is sampled between 50% and 200%. The cap represents an assumed willingness of arbitrage buyers to buy discounted principal near maturity. With 60 days remaining and 100% stressed APY, the cap is 1 - 2^(-60/365) ≈ 10.77%.

## Why the graph has no circular market link

The PT points to another market as a liquidation venue. That venue points to its oracle and protocol; it does not point back to the PT as collateral. This keeps the computational graph acyclic. The venue itself has no standalone liquidity-drain mode, because a run on its lenders is not directly a loss for our PT holding.

This is a one-pass fire-sale approximation, not an order-book simulation or iterative liquidation cascade. The assumed arbitrage cap may fail when backing is uncertain or capital is scarce. A mature PT is assigned no fire-sale dip here; that does not prove redemption or issuer safety.

12 / EXIT MODEL

# What an exit really costs

## Held PT: select a route at our size

The engine asks KyberSwap and Pendle for available PT-to-cash routes and chooses the cheaper quoted cost. The destination is USDC or USDT, excluding the held asset and its discovered transitive backing. Selling a USDC-backed PT into USDC does not remove its USDC backing risk.

**proceeds USD = raw output / 10^token decimals × independent output-token USD price**

**quoted cost = max(0, 1 - (proceeds USD - gas USD) / sale size USD)**

The independent mark comes from DefiLlama; USDC and USDT are not assumed worth $1. Pendle fees are already included in route output and are not added again. Pendle gas is explicitly estimated as $40; the Kyber route uses its reported gas estimate.

## Market supply: withdrawal comes before selling

**calm withdrawable fraction = clip(idle liquidity USD / position USD, 0, 1)**

Under modeled stress, other lenders take a share of idle liquidity first: share ~ Beta(2, 2.5), with mean about 44%. Withdrawal gas is modeled at $40. If the loan token itself is damaged, the withdrawn portion must also be sold. Stress price impact is widened by a lognormal multiplier with median 2 (mean about 2.27).

## Fallback estimate is not a verified route

**constant-product approximation: impact = sale size / (pool depth / 2 + sale size)**

For a $1M sale into $20M total depth, impact is 1 / (10 + 1) = 9.09%. Pendle is not literally a symmetric constant-product dollar pool; this is a fallback / fire-sale approximation. Without a confirmed complete held-PT route, f becomes zero, so no immediate loss avoidance is credited.

Quotes are indications, not executable guarantees. Cache age, MEV, pool changes, transaction failure, output-asset risk and actual execution can change realized costs. The model is not a transaction simulator.

13 / THE STAY / EXIT DECISION

# Two decision rules: scenario and daily

## Single-component scenario

**EXIT if p × E[L] × f > c**

This compares the modeled loss avoided in one selected failure scenario with the whole-position exit cost. The cost includes slippage, gas and a default seven days of forgone yield while redeploying. The scenario does not subtract a separate full-horizon yield credit.

**scenario break-even probability = c / (E[L] × f)**

## Latest daily monitoring calculation

**E = sum over i of p_i × E[L_i]**

**Y = current APY × max(H - redeploy days, 0) / 365**

**EXIT if net avoidable loss = (E - Y) × f > c**

Daily monitoring includes base-rate risk and signal updates, but now credits the yield given up by leaving for the rest of the horizon. c already charges the seven-day redeployment carry, so Y only covers the remaining days. Exited capital is assumed to earn nothing afterwards. Yield is approximated linearly even though the input is named APY.

At H = 30 and redeployment = 7 days, c includes seven days of yield and Y credits the other 23. Together they represent the modeled whole-horizon yield cost of leaving. A negative net-loss number means yield exceeds modeled expected loss under this assumption; it is not a forecast of guaranteed profit.

Daily and scenario results can differ without a software contradiction: their risk aggregation, liquidity stress and yield treatment differ. The daily component “break-even crossing” event still uses c / (L × f), not the full aggregate yield-adjusted boundary. Do not present that event as the daily verdict's exact trigger.

13 / THE STAY / EXIT DECISION

# Read the output without overclaiming

| Output | Correct interpretation |
|---|---|
| Probability and modeled range | A conditional model estimate over the selected horizon. The range is not a certified forecast-accuracy interval. |
| Average / typical / bad case | Mean / median / 95th percentile of conditional position loss. These are not three independently validated predictions. |
| Expected loss USD | p × average conditional loss × position size in a scenario. It is not the worst possible loss. |
| Book impact | Expected loss relative to entered book size; no full correlated multi-investment book simulation. |
| Confidence / agreement | For model-based scenarios: the share of sampled probability/cost comparisons supporting the verdict. It is not the probability the recommendation is correct. |
| Withdrawable share | How much modeled exposure can be removed now. The remainder stays exposed. |
| STAY with blocked route | An exit was not demonstrated to be feasible. Do not interpret this as a positive endorsement of safety. |
| REVIEW | Required input data missing/invalid. Inspect issues and obtain fresh evidence before relying on the numeric decision. |

The scenario agreement calculation varies probability and cost while holding mean loss and mean withdrawable share fixed. It therefore understates some uncertainty. When probability is manually overridden, the backend uses a distance-to-break-even heuristic; the frontend labels it as a fixed input instead of calibrated decision confidence.

The loss histogram shows simulated conditional loss. The highest display bucket clips at roughly the 99.5th percentile, so the visible tail is not a complete worst-case bound. AI text is secondary: verify the numeric assumptions and propagation paths first.

> “No active alerts” means the implemented rules did not trigger on the available readings. It does not mean no exploitable bug exists, no omitted dependency matters, or that a catastrophe is impossible.

14 / MEMORY AND LEARNING

# History, notifications and calibration

SQLite stores recorded runs, per-component forecasts, signals, raw metrics, observed outcomes, notifications, saved watchlists and schedule settings. Interactive graphs remain in process memory; browser workspaces have separate persistence. These three stores serve different purposes.

| Saved change | What happens |
|---|---|
| Verdict flips | Event records the old/new verdict and decision quantities. |
| Component crosses break-even | Warning or recovery event; see the boundary limitation on page 23. |
| Alert rises / falls | Stored change with signal reasons. |
| Expected loss doubles | Trend warning if the absolute change is also material. |
| Required data disappears / returns | Data-quality event. Incomplete runs are omitted from calibration forecasts. |

## Calibration asks whether probabilities match frequencies

**Brier score = average((forecast probability - observed outcome)^2)**

For a forecast p = 0.1, no failure gives score 0.01; a failure gives 0.81. Lower average score is better. The report compares with a climatology baseline, calculates log loss, and groups forecasts into probability bins: do predictions near 5% fail about 5% of the time?

Only matured horizons are scored. Daily duplicates are removed; non-overlapping windows per component and mode across positions are selected for scoring. Distinct incidents are counted separately from positive forecasts. The report retains a low-data warning below ten distinct incidents. Different components may still be correlated.

Outcomes must be recorded comprehensively. An absent outcome is currently interpreted as no failure, not as unknown. A single incident covered by many forecasts is not many independent failures. Empirical likelihood ratios are reported for inspection; the system does not automatically retrain or replace its hand-set parameters.

15 / WORKED ARITHMETIC

# Work the core decision yourself

These four cases are exact arithmetic examples, verified against decide(). They are not live quotes. Position V = $1,000,000; book B = $10,000,000. All rates below are fractions of the whole position unless marked otherwise.

| Case | Inputs | Calculation | Verdict |
|---|---|---|---|
| A: liquid, severe risk | p 30%; loss 50%; f 100%; c 11% | Avoidable loss 15% = $150,000; exit $110,000 | EXIT |
| B: smaller risk | p 10%; loss 40%; f 100%; c 11% | Avoidable loss 4% = $40,000; exit $110,000 | STAY |
| C: half withdrawable | p 30%; loss 50%; f 50%; c 11% | Avoidable loss 7.5% = $75,000; exit $110,000 | STAY |
| D: route blocked | p 90%; loss 50%; f 0%; c 0% | Avoidable loss 0; route unavailable, exposure remains | STAY + blocked |

## Break-even in A versus C

**A: 0.11 / (0.50 × 1.00) = 22%**

**C: 0.11 / (0.50 × 0.50) = 44%**

The same failure probability of 30% is above A's boundary but below C's. The withdrawable fraction changes the value of exiting. Removing half the position does not remove the other half's loss.

## Do not confuse dollars and percentages

If exit cost is quoted as a percentage of the withdrawn amount, convert it to a whole-position percentage before comparing. At f = 50%, a 10% cost on the withdrawn half is 5% of the original position, not 10%. The engine scales sale/carry costs to the whole position and adds fixed gas separately.

A break-even probability over 100% means the selected exit costs more than the modeled loss even if failure were certain. At f = 0, there is no useful immediate-exit boundary.

15 / WORKED ARITHMETIC

# Work the latest daily yield rule

Assume a $1M lending position, f = 100%, current supply APY = 8%, horizon = 30 days, redeployment = 7 days, $40 withdrawal gas and no loan-token swap. These are teaching numbers.

**seven-day carry = 8% × 7 / 365 = 0.15342%**

**c = $40 / $1M + 0.15342% = 0.15742% = $1,574**

**Y = 8% × 23 / 365 = 0.50411% = $5,041**

| Daily risk E | Net avoidable loss (E - Y) × f | Compare with c | Verdict |
|---|---|---|---|
| 0.40% = $4,000 | -$1,041 | Below $1,574 | STAY |
| 1.00% = $10,000 | $4,959 | Above $1,574 | EXIT |

## What changes if only half can leave?

At f = 50%, using E = 1%, net avoidable loss = (1% - 0.50411%) × 50% = 0.24795% of the whole position, or $2,479. Carry on the withdrawn half is $767; with $40 gas, c is about $807. That particular set of numbers still says EXIT for the available half. The rest remains exposed.

## Why both yield terms are present

The exit-cost calculation includes seven days of redeployment carry; the daily yield credit covers the remaining 23 days. Putting 30 days into Y as well would double-charge the first seven days. The current code avoids that in the usual H ≥ 7-day case.

This is a counterfactual assumption, not a measured alternative strategy. The model does not compare yields from another real investment, compound the APY, or reduce promised yield according to failure-dependent payments. For H under seven days, the exit model still charges its default seven-day redeployment carry.

16 / LIVE SNAPSHOT

# A checked live example from vinkas

Read-only capture of the built-in PT-reUSD/USDC example on **11 October 2026 at 00:39:54 Zurich time**. Entered size $1M, entered book $10M, horizon 30 days. This standalone check had no stored monitoring history, so it used current signals without historical contagion events.

| Observed / computed quantity | Checked value |
|---|---|
| Graph | 17 nodes / 20 edges; no discovery warnings |
| Primary market LLTV | 91.5% |
| Primary market utilization | 88.55% |
| Idle liquidity | $13,823,112 |
| Current supply APY | 7.33% |
| Pendle PT-pool liquidity | $12,411,999 |
| Other modeled PT collateral | $10,198,380 in PT-reUSD/USDT venue |
| Feed update age at capture | 6 seconds |
| Loan token mark | USDC about $0.999712 |

| Daily decision quantity | Value on $1M |
|---|---|
| Total modeled expected loss | 0.55% / $5,530 |
| Remaining-horizon yield credit | 0.46% / $4,621 |
| Net avoidable loss | 0.09% / $909 |
| Exit cost including redeployment carry | 0.14% / $1,446 |
| Verdict / data status | STAY / complete; no current rule alerts |

The snapshot demonstrates a real working pipeline, not forecast calibration. APY, LLTV, liquidity, graph structure and verdict can change. Some preset descriptions still use older LLTV values; use the observed market attributes.

17 / DEMO CASES

# Three reproducible demonstrations

These cases were checked on the captured live graph with 5,000 scenario draws. They did not send trades or call an LLM. The first two deliberately override probability and exit cost to make the decision boundary easy to explain.

| Case | Inputs and result | What it demonstrates |
|---|---|---|
| 1 EXIT | Select USDC / depeg. Set probability 80%, exit cost 0.5%. Conditional loss ≈ 5.96%; withdrawable share ≈ 99.77%. Avoidable loss ≈ $47,602 > $5,000. | A loan-token failure directly affects dollar value of the lending claim. |
| 2 STAY | Same component/mode, probability 5%, exit cost 11%. Avoidable loss ≈ $2,975 < $110,000. | Same dependency, different economics. A warning does not force an exit. |
| 3 Oracle scenario | Select primary oracle / wrong price; manual alert; no numeric overrides. p ≈ 1.585%; mean conditional position loss ≈ 1.185%; expected loss ≈ $188. Modeled exit ≈ $1,443; STAY. | Buffers, probability and exit cost matter together. |

## Additional controlled scenarios worth discussing

**Blind oracle:** compare a tracking versus non-tracking oracle in an offline synthetic market after a large collateral drop. Non-tracking usually leaves more bad debt. **Sibling venue:** an underpricing oracle can fire-sell the PT through its AMM. **Data failure:** an unreadable feed yields REVIEW, not healthy STAY. **Liquidity loss:** lower available cash and inspect f and residual exposure.

The UI cannot directly edit all raw graph attributes or arbitrary severity. Those extra cases are covered by offline tests and explanations; do not claim they are one-click live-data controls. Scenario results can vary when live liquidity changes.

17 / DEMO RUNBOOK

# A practical 7-minute demonstration

| Time | Action | What to say |
|---|---|---|
| 0:00-0:45 | Open the PT-reUSD/USDC preset and refresh. | We entered a USDC lending position. Borrowers' PT collateral creates dependencies we do not directly hold. |
| 0:45-1:45 | Trace market / PT / backing, oracle, AMM and chain. Inspect one source. | Graph arrows show dependency. Loss travels back through the economic relationships. Evidence and assumptions are distinguishable. |
| 1:45-3:00 | Select USDC, depeg, Advanced overrides: p 80%, c 0.5%. Run scenario. | This is a controlled demonstration. At these inputs, avoidable modeled loss exceeds the modeled exit cost. Show the arithmetic and exit fraction. |
| 3:00-4:00 | Repeat with p 5%, c 11%. | The graph is unchanged, but the decision is STAY because exiting costs more than avoided loss. |
| 4:00-5:00 | Clear overrides. Inspect oracle wrong-price scenario and assumptions. | An oracle problem is not automatically a lender loss. LLTV and liquidation transmission matter. |
| 5:00-6:15 | Open Scheduled STAY / EXIT. Inspect latest run, drivers and history. | This is polling and recording, not incident replay. Daily decisions also account for current yield. |
| 6:15-7:00 | Show an assumption and a limitation, then conclude. | Our contribution is traceable decision support on real data, not guaranteed forecasting or automatic execution. |

**Before presenting:** start the latest branch, check health, refresh the preset, verify data status and rehearse both override cases. Allow time for protocol APIs. If real quotes change, explain the formula rather than promise the same number. For a live recorded run, add only the intended investment, check watchlist source and use Run now; leave the schedule paused after a demo unless deliberately needed.

Historical replay applies a catalogue incident's approximate severity to a comparable component in today's graph. It is a counterfactual stress test, not an exact reconstruction using historical blockchain state.

18 / OPERATIONS

# Run, recover and troubleshoot

| Task | Procedure / interpretation |
|---|---|
| Start locally | From the repository root: npm run dev. API normally uses port 8000; Vite normally uses 5173 and proxies /api. Setup requires uv and Node; frontend tests require Node 22.6+. |
| Serve built UI | npm start builds the frontend and serves it through the backend. Backend process and network access must remain available. |
| No AI provider | Template explanation works without a key. An AI summary is optional and does not gate calculations. |
| Unknown position ID | Refresh/reanalyze: interactive graphs are in memory and disappear on restart. SQLite monitoring history is separate. |
| REVIEW / failed run | Inspect missing fields, source errors, discovery warnings and timestamp. Recover the source or refresh; an older saved verdict is not a new successful check. |
| No exit route | Do not force f to one. A quote estimate without a complete route is not proof of execution. Investigate liquidity and routing. |
| Schedule not running | Check the interval, selected watchlist, backend process and last/next run. Paused means interval 0. |
| Offline / rate limited | Live discovery can fail or be partial. Use a clearly labeled saved example / offline test to explain methodology, not as live evidence. |

**Live does not mean tick-by-tick:** market/quote data commonly has 120-second caches; RPC batches about 300 seconds; Morpho token metadata about 600 seconds; Pendle pool/asset metadata about 900 seconds; protocol TVL about 3,600 seconds. A five-minute schedule does not make every input five minutes fresh or all observations atomic.

For a controlled internal prototype, the current API is convenient. It has no application-level user authentication, permission model or production-grade deployment supervision. Restrict access and treat configuration, outcome entry and optional external AI/webhooks as operational controls before daily institutional use.

19 / DEFENSIBLE CLAIMS

# What makes the project good - honestly

| Strength | What supports it | What it does not prove |
|---|---|---|
| Economic usefulness | Costs and withdrawable exposure enter the decision, not just a red flag. | The best real trade or risk-adjusted strategy is not optimized. |
| Traceability | Evidence, component modes, equations and propagation paths are visible. | Every issuer / oracle classification is correct. |
| Real implementation | Live protocol discovery, size-specific routes, SQLite monitoring and working UI. | High availability or executable price guarantees. |
| Non-linear lending logic | Collateral buffers, blind oracles, over-borrowing and sibling fire sales. | Exact borrower-level liquidation / correlated cascade forecasting. |
| Uncertainty made visible | Simulated distributions, ranges, assumptions and REVIEW for missing required inputs. | Comprehensive uncertainty or externally calibrated probabilities. |
| Testing discipline | 66 Python tests and 21 frontend tests passed on this version; production build passed. | Profitability, security audit or proof of forecast skill. |

## What was specifically checked for this manual

The latest branch was fetched and its core sources inspected. The live PT-reUSD market was analyzed without recording operational history. Controlled loan-token EXIT and STAY cases and an oracle case ran against that graph. The tests cover decision arithmetic, oracle ratios, exit-output valuation, partial/blocked exits, signals, data quality, history, calibration and collateral venues.

The production build passed with an existing large-bundle warning. The checked snapshot and examples are tied to commit e85961f; UI styling can change without changing these methods, but any backend model change requires updating the manual and rerunning the demonstrations.

A defensible pitch: “We turn a position's hidden dependency stack into a transparent loss-and-exit comparison on real data, with operational monitoring and explicit assumptions.” Do not say “We predict hacks accurately” or “We have eliminated correlation.”

20 / REFERENCE

# Reference map and rehearsal checklist

| Read the implementation only if needed | What it anchors |
|---|---|
| engine.py / decision.py | Orchestration; daily yield credit; scenario comparison; outputs. |
| discovery.py / graph.py | Position constraints, dependency edges, evidence and venue limits. |
| probability.py / failures.py | Gamma-Poisson assumptions, likelihood ratios, mode/asset priors. |
| signals.py / contagion.py | Thresholds, rule families and time-varying multipliers. |
| propagation.py / exit_cost.py | Non-linear lender losses, fire sales, execution assumptions. |
| history.py / calibration.py / notify.py | Persistent records, scoring, event comparison and delivery. |
| backend/app/api.py / frontend components | Scheduling, API validation, inspector and result views. |

**Source version:** ezhoureal/belem, commit e85961f. This checked-in source is the authority for this manual's implementation claims. README and CLAUDE.md provide context but some introductory labels simplify the actual equations.

**[P1] Pendle, Principal Token mechanics:** PT denomination and maturity redemption.
**[P2] Morpho, Liquidation:** LTV, LLTV and liquidation mechanics.
**[P3] NIST Engineering Statistics Handbook:** Gamma distribution; exponential distribution. Protocol/math references were checked for this document; parameters remain prototype judgments.

## You are ready when you can explain

The difference between holding a PT and lending USDC against it; one complete dependency path; p versus conditional loss; Gamma-Poisson and odds updates; the LLTV buffer; why f matters; both decision rules; how the scheduler remembers results; and three limitations without hiding behind technical terms.

The following final pages are the critical questions. Practice the short answer aloud, then explain the mathematical or operational boundary if challenged.

21 / DEFEND THE PROJECT

# Critical questions 01-04

## 01. Why not just show a risk score or a red alert?

Because the employee must choose an action. We connect a specific failure to dollars lost, the portion that can be removed, and the cost of removing it. An alert is an input to the probability update, not the decision itself. The trade-off is inspectable.

## 02. Is this a hack predictor, and have you shown it makes money?

No. We have shown a functioning decision-support pipeline on live data and tested its internal behavior. Forecast accuracy and realized trading performance have not been established. We would need forward outcome records, counterfactual execution data and out-of-sample evaluation to make those claims.

## 03. What is actually live, and what did you invent?

Market balances, oracle reads, pool metadata and route outputs are requested from live sources with caches. Failure rates, severity priors, likelihood ratios, leverage ranges, contagion weights, stress widening and arbitrage caps are configured judgments. Demo overrides are explicitly fixed inputs. A live-data interface is not a fully data-estimated risk model.

## 04. How do you know the dependency graph is complete?

We do not. It follows discovered contracts and protocol APIs plus registry/heuristic links, with bounded recursion and only the largest five qualifying Morpho collateral venues. Off-chain issuer exposures, other protocols and undiscovered controls can be missing. Evidence labels and warnings expose some gaps; graph absence is not proof of no risk.

21 / DEFEND THE PROJECT

# Critical questions 05-08

## 05. Where did k and T come from? Are they an unbiased incident sample?

They are class-level historical judgments. The small incident catalogue does not establish the component-year denominators or correct selection, survivorship and reporting biases. Gamma-Poisson provides a coherent positive-rate model conditional on those inputs, not a statistical seal of approval for them. Calibration and better exposure accounting are the next evidence needed.

## 06. Why Poisson if failures cluster?

Poisson is the simple starting model for arrivals at a given rate. The contagion multiplier makes that rate vary after recent events, relaxing a constant-rate assumption heuristically. It is not a fully estimated self-exciting process; the weights and decay are hand-set and the dependence can still be wrong.

## 07. Why Gamma and Beta rather than a machine-learning model?

Gamma represents uncertainty in a positive rate and has a natural event-count interpretation. Beta respects the zero-to-one bounds of losses and shares. They give an understandable baseline with sparse labels. A trained model would need much stronger data coverage and could still miss new exploits. We use AI for optional explanation, not numerical prediction.

## 08. Does a 90% modeled range contain the truth 90% of the time?

That has not been demonstrated. It is a 5th-to-95th percentile range conditional on our priors and model structure. Missing causes, parameter bias and correlated incidents are not all inside it. Frequentist coverage must be evaluated with repeated matured forecasts and reliable labels.

21 / DEFEND THE PROJECT

# Critical questions 09-12

## 09. Are the signal likelihood ratios measured? What makes two signals independent?

The ratios are configured assumptions. Independence is approximated through rule families: price drop and APY jump share a family, for example. Distinct families need not be statistically independent. We prevent known duplicate causes such as a stale feed and its parent warning, but the remaining update can still overstate evidence.

## 10. If an incident is confirmed, why use 85-99% instead of 100%?

That is the implementation's incident-analysis convention, not a measurement of whether a documented past event happened. Replay uses it to exercise the decision under high incident probability while severity is anchored to the catalogue. We must label that convention and avoid interpreting it as historical uncertainty about an established event.

## 11. Do you double-count risks or correlations?

Some known duplication is removed: the feed warning and inherited parent warning share a cause, and wrapper discounts are separated from underlying depegs. However, the screening sum and product for any-event probability still simplify correlation; common issuers, chains and overlapping modes are not a complete joint-loss model. We do not claim portfolio-wide correlation has been solved.

## 12. How can you call an oracle non-tracking just from its description?

We cannot prove that classification from a label. The code uses feed-description and contract-structure heuristics to distinguish market feeds from reference/NAV rates. This can misclassify a bespoke oracle. The operator should inspect the evidence; validating the actual response to market movements would materially strengthen this dependency classification.

21 / DEFEND THE PROJECT

# Critical questions 13-16

## 13. Is an oracle-versus-DEX comparison actually independent?

The current price-gap input compares the Morpho oracle price with a collateral/loan ratio using Morpho token USD marks. It is not an independent raw DEX time series. The variable name is broader than the measurement. In addition, automatic detection only flags overpricing; underpricing venue fire sales are modeled but not comprehensively signaled.

## 14. What happens if a critical feed is unreadable or updates every hour?

Missing/invalid answers or ages produce REVIEW and suppress calibration forecasts. A readable feed is stale only under the current configured thresholds: 26 hours normally, 35 days for identified NAV feeds. Those thresholds can be inappropriate for a one-hour heartbeat. Complete data means required fields exist, not that every freshness policy is correct.

## 15. Why can a healthy-looking market receive EXIT without an alert?

Base-rate loss is still real modeled risk; the latest daily comparison includes it and credits current yield. If total expected loss net of yield exceeds the exit cost, EXIT can follow without a new alert. We should investigate assumptions and execution, not alter probability merely to force a calm STAY.

## 16. Why can the scenario and daily screen disagree?

The scenario isolates one mode and uses stressed exit assumptions plus a short redeployment carry. Daily monitoring aggregates component-mode expected losses, uses calm exit pricing and credits remaining-horizon yield. They answer different comparisons. The component break-even event is still a gross single-mode threshold, so it is not exactly the aggregate daily yield boundary.

21 / DEFEND THE PROJECT

# Critical questions 17-20

## 17. Is the yield credit a guarantee, or does it ignore alternative returns?

Neither the input APY nor continued payment is guaranteed. The code linearly credits the current rate and assumes exited capital earns nothing after redeployment. It does not model an actual alternative investment or failure-dependent yield loss. This assumption can materially favor staying and should be stress-tested rather than presented as optimal asset allocation.

## 18. Why not just exit the whole position when the model says EXIT?

Because f can be below one. The comparison only credits loss avoidance on what can be removed now. Costs are scaled to the whole position and fixed gas remains. The unwithdrawable remainder remains exposed. With no confirmed held-PT cash route, f is zero and the result must be explained as execution blocked, not safety.

## 19. Could a swap into USDC still expose us to the same problem?

Yes. The route-selection code excludes the held token and transitive backing addresses from cash destinations, reducing an obvious retained-backing problem. It does not prove the output issuer, settlement chain, exchange or custody arrangement is independent of the incident. Output-token risk remains a residual exposure.

## 20. Is choosing the cheaper quote proof we can execute at that price?

No. It chooses between available quoted routes at size and values raw outputs independently; it does not submit transactions. Quote freshness, MEV, pool movement, failed swaps, approval requirements and stress can change execution. Gas and stress widening are partly modeled assumptions. A production exit requires separate transaction simulation and controls.

21 / DEFEND THE PROJECT

# Critical questions 21-24

## 21. Is the Pendle pool really constant-product, and is your price cap reliable?

The fallback/fire-sale impact uses a symmetric constant-product approximation, not Pendle's exact curve. The maturity cap assumes buyers accept 50-200% annualized return and have capital available. That cap is questionable if backing fails or redemption is frozen. It is a transparent approximation to sensitivity, not a hard economic bound.

## 22. Does this simulate a complete liquidation cascade?

No. It adds a single-pass sale channel from selected other Morpho collateral venues into the PT pool, with sampled sold fractions and a cap. It does not iterate feedback among all markets, solve market-clearing prices, or reconstruct all borrower positions. The graph remains acyclic partly to keep the approximation tractable.

## 23. Does the confidence figure mean the decision is correct?

No. For a modeled scenario it is sample agreement as probabilities and costs vary, while average conditional loss and liquidity share are held fixed. For a fixed probability the backend uses a heuristic, and the UI labels the override. It is a robustness cue within assumptions, not an empirically calibrated success probability.

## 24. Why keep the random seed fixed? Are we hiding uncertainty?

The seed makes identical inputs produce repeatable sampled calculations and reduces noise in comparisons. Distributions and their ranges are still shown. Changes in live inputs or alerts can change the result. The fixed seed does not validate the model, and changes in graph topology or sample order can still affect numerical comparisons.

21 / DEFEND THE PROJECT

# Critical questions 25-28

## 25. What does replay prove, and what does it not prove?

It tests whether our propagation and decision respond sensibly to a catalogue shock. The historical severity is applied to a comparable component in today's graph, with noise and high incident probability. It is not a backtest with historical positions, liquidity, oracles and feasible execution. Passing replay is not proof of predictive skill.

## 26. Can calibration claim success after one event appears in 31 forecasts?

No. Distinct incidents are separate from positive forecasts; non-overlapping windows per component/mode across positions are used for scoring. The low-data warning remains below ten distinct incidents. Different components still correlate, and incomplete outcome recording biases scores toward no failure. The reported empirical ratios do not automatically retrain the model.

## 27. Can this be deployed for institutional daily use immediately?

It can be used as a supervised prototype, not represented as a finished institutional service. The API has no built-in authentication, the schedule depends on one process, sources can be cached/unavailable, and classification/calibration gaps remain. Monitoring history persists, but interactive graph state does not. Deployment needs access controls, supervision, data-quality policy and execution governance.

## 28. What exactly is your defensible innovation, and what should judges take away?

The integration is the contribution: discover a multi-layer position from real data, propagate failure through economic relationships, value a realistic-size exit, and present a transparent decision with history. We can demonstrate EXIT, STAY, partial/blocked execution and REVIEW without claiming guaranteed returns. The strongest evidence is the reproducible chain from a source observation to an explicitly qualified decision.